

Engineering Document E13 — Executive Prompt Framework & Prompt Composition Engine

Product Name: HQ

Engineering Package: Version 1.0

Purpose

This document defines the Prompt Composition Engine that powers every AI Executive in HQ.

Instead of maintaining large static prompts, HQ dynamically assembles prompts from reusable modules based on the mission, organization, user, executive, memory, and available tools.

This creates a scalable, maintainable, and intelligent prompting architecture.

Vision

Prompts are not documents.

They are **software components**.

Every prompt is generated specifically for the current mission.

No unnecessary information is included.

No executive receives irrelevant context.

Core Principles

The Prompt Engine must be:

- Modular
 - Context-aware
 - Token-efficient
 - Versioned
 - Testable
 - Secure
 - Extensible
-

Prompt Composition Pipeline

Mission

↓

CEO Planning

↓

Executive Selection

↓

Context Collection

↓

Memory Retrieval

↓

Tool Discovery

↓

Prompt Assembly

↓

LLM Execution

↓

Structured Output

↓

CEO Review

Every prompt follows this pipeline.

Prompt Architecture

Every executive prompt consists of reusable modules.

Identity
+
Mission
+
Responsibilities
+
Organization Context
+
User Context
+
Mission Context
+
Relevant Memory
+
Available Tools
+
Collaboration Context
+
Output Schema
+
Guardrails
=

Module 1 — Executive Identity

Defines:

- Executive name
- Department
- Role
- Authority
- Personality
- Communication style

Example:

"You are the Marketing Director of HQ."

Module 2 — Mission

Defines:

- Current objective
 - Success criteria
 - Expected deliverables
 - Constraints
 - Priority
-

Module 3 — Organization Context

Provides:

- Organization profile
- Industry
- Brand values
- Headquarters configuration

- Business goals

Executives understand the organization before making decisions.

Module 4 — User Context

Includes:

- User preferences
- Language
- Tone
- Previous interactions
- Active organization role

Only relevant information is injected.

Module 5 — Mission Context

Contains:

- Mission ID
 - Background
 - Current progress
 - Existing deliverables
 - Deadlines
 - Dependencies
-

Module 6 — Memory Context

Retrieves only relevant memories.

Sources:

- Organization Memory
- Executive Memory

- Mission Memory
- User Preferences
- Historical Decisions

Irrelevant memories are excluded.

Module 7 — Collaboration Context

Executives receive:

- Previous executive outputs
- CEO instructions
- Pending reviews
- Outstanding blockers

This keeps collaboration synchronized.

Module 8 — Tool Context

Executives are informed of available tools.

Examples:

- Search
- Calculator
- File Analysis
- Code Generator
- Image Generator
- Reporting
- Analytics

Executives only see tools they are authorized to use.

Module 9 — Output Schema

Every executive returns structured data.

Example:

Summary

Findings

Recommendations

Risks

Confidence

Next Actions

This simplifies downstream processing.

Module 10 — Guardrails

Guardrails include:

- Stay within assigned role
- Respect permissions
- Avoid unsupported claims
- Escalate uncertainty
- Follow organization policies
- Return structured output

These ensure consistent behavior.

Prompt Versioning

Every module includes:

- Version

- Created Date
- Updated Date
- Author
- Status

Prompt changes are tracked over time.

Prompt Registry

The Prompt Registry stores:

- Module definitions
- Executive templates
- Prompt versions
- Output schemas
- Validation rules

The registry becomes the central management system.

Dynamic Context Injection

Before execution, the engine selects only the context needed.

Priority order:

1. Mission Context
2. Organization Context
3. Executive Memory
4. User Preferences
5. Collaboration History
6. Tool Availability

Reducing unnecessary context lowers token usage and improves response quality.

Prompt Validation

Before sending to the LLM, prompts are validated for:

- Missing modules
- Invalid references
- Empty context
- Unsupported tools
- Token budget

Invalid prompts are rejected before execution.

Token Budget Management

The engine assigns a token budget.

Priority:

Mission

↓

Critical Memory

↓

Organization Context

↓

Executive Context

↓

Optional Context

Lower-priority modules are trimmed first if limits are reached.

Multi-Provider Support

The Prompt Engine supports:

- Gemini
- OpenAI
- Anthropic
- Future providers

Prompt composition remains provider-independent.

Prompt Testing

Each prompt module is tested for:

- Completeness
- Role consistency
- Output quality
- Guardrail compliance
- Token efficiency

Automated tests ensure reliable behavior.

Prompt Analytics

Track:

- Prompt version usage
- Token consumption
- Execution time
- Success rate
- Revision rate
- User satisfaction

These metrics guide prompt improvements.

Security

The Prompt Engine must:

- Remove unauthorized data
- Prevent prompt injection where possible
- Validate external inputs
- Restrict sensitive context
- Log prompt execution metadata

Sensitive organizational information is never exposed unnecessarily.

Future Evolution

The architecture supports:

- Industry-specific prompt packs
 - Enterprise prompt customization
 - A/B prompt testing
 - Automatic prompt optimization
 - Self-improving prompt modules
 - Multi-language prompt generation
-

Success Criteria

The Prompt Composition Engine succeeds when:

- Prompts are modular and reusable.
 - Executives receive only relevant context.
 - Token usage remains efficient.
 - Prompt updates require minimal effort.
 - Outputs remain consistent across executives and providers.
-

Conclusion

The HQ Executive Prompt Framework replaces static prompts with a dynamic Prompt Composition Engine.

By assembling prompts from reusable, versioned modules tailored to each mission, HQ achieves better maintainability, stronger consistency, lower token costs, and greater flexibility. This architecture positions HQ to scale from a hackathon prototype into a sophisticated multi-agent AI platform without relying on fragile prompt engineering practices.